

Documento de Diseño

Sistema de Análisis Algorítmico Financiero

Universidad del Quindío

Programa de Ingeniería de Sistemas y Computación

Curso: Análisis de Algoritmos — 2026-1

Generado: 12/05/2026 03:37

Documento de Diseño — Sistema de Análisis Algorítmico Financiero

Universidad del Quindío — Programa de Ingeniería de Sistemas y Computación

Curso: Análisis de Algoritmos — 2026-1

1. Introducción

El análisis financiero moderno depende de la capacidad computacional para procesar grandes volúmenes de datos históricos y detectar patrones relevantes en el comportamiento de activos financieros. Este proyecto implementa un sistema que aplica métodos cuantitativos, algoritmos clásicos y técnicas de análisis de series temporales sobre datos reales de la Bolsa de Valores de Colombia (BVC) y ETFs internacionales (S&P; 500).

El sistema se compone de cinco módulos principales:

1. **ETL:** Extracción, limpieza y unificación automatizada de datos financieros
2. **Similitud:** 4 algoritmos de comparación entre series temporales
3. **Patrones y Volatilidad:** Detección de patrones por ventana deslizante y clasificación de riesgo
4. **Dashboard:** Visualizaciones interactivas y reporte PDF exportable
5. **API REST:** Despliegue como aplicación web con documentación completa

2. Arquitectura del Sistema

2.1 Estructura del Proyecto

```

investment-algorithm/
■■■ src/
■   ■■■ __init__.py
■   ■■■ api/
■   ■   ■■■ __init__.py
■   ■   ■■■ gateway.py           # App factory + registro de Blueprints
■   ■   ■■■ routes/
■   ■   ■   ■■■ __init__.py
■   ■   ■   ■■■ similarity.py    # Req 2: endpoints de similitud
■   ■   ■   ■■■ patterns.py     # Req 3: endpoints de patrones y volatilidad
■   ■   ■   ■■■ dashboard.py    # Req 4: candlestick, SMA, resumen
■   ■   ■   ■■■ reports.py      # Req 4: generación de PDF
■   ■   ■■■ templates/
■   ■   ■■■ base.html           # Layout base con navegación
■   ■   ■■■ pages/
■   ■   ■■■ index.html          # Landing con estadísticas
■   ■   ■■■ similarity.html     # Comparación interactiva de similitud
■   ■   ■■■ patterns.html      # Detección de patrones
■   ■   ■■■ risk.html           # Ranking de riesgo por volatilidad
■   ■   ■■■ dashboard.html     # Dashboard completo + exportación PDF
■   ■■■ etl/
■   ■   ■■■ __init__.py
■   ■   ■■■ fetcher.py          # Extracción HTTP directo (Yahoo Finance)
■   ■   ■■■ scraper.py          # Scraper alternativo (fallback)
■   ■   ■■■ cleaner.py          # Limpieza: duplicados, outliers, interpolación
■   ■   ■■■ unifier.py          # Unificación, alineación de calendarios
■   ■■■ services/
■   ■   ■■■ similarity/         # Req 2: 4 algoritmos de similitud
■   ■   ■   ■■■ __init__.py    # SimilarityAnalyzer (agregador)
■   ■   ■   ■■■ euclidean.py   # Distancia Euclidiana  $O(n)$ 
■   ■   ■   ■■■ pearson.py     # Correlación de Pearson  $O(n)$ 
■   ■   ■   ■■■ dtw.py         # Dynamic Time Warping  $O(n \times m)$ 

```

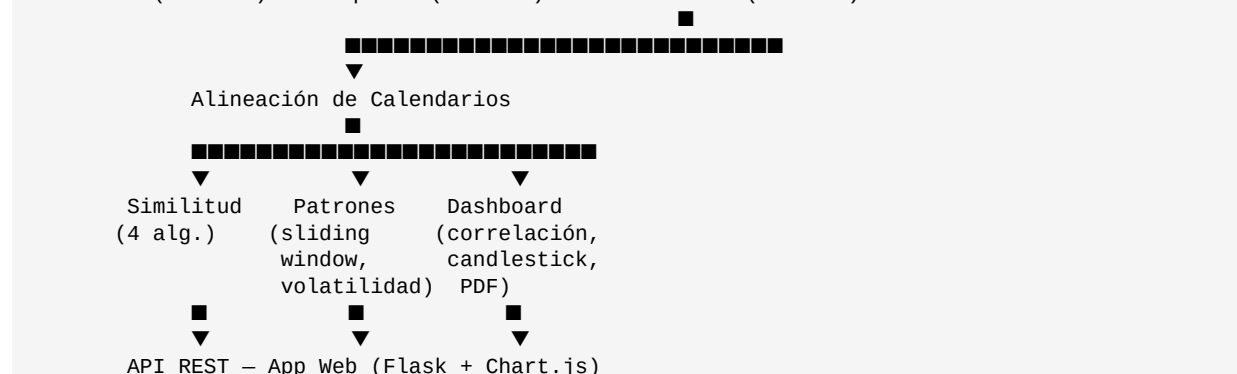
```

■ ■ ■ ■ ■ cosine.py           # Similitud por Coseno 0(n)
■ ■ ■ ■ ■ patterns/           # Req 3: patrones y volatilidad
■ ■ ■ ■ ■ __init__.py
■ ■ ■ ■ ■ sliding_window.py    # Ventana deslizante 0(n)
■ ■ ■ ■ ■ volatility.py        # Volatilidad y riesgo 0(n)
■ ■ ■ ■ ■ reporting/          # Req 4: reportes
■ ■ ■ ■ ■ __init__.py
■ ■ ■ ■ ■ pdf_report.py        # Generación de PDF con ReportLab
■ ■ ■ ■ ■ technical.py         # SMA e indicadores técnicos 0(n)
■ ■ ■ ■ ■ volume_analyzer.py    # Análisis de volumen
■ ■ ■ ■ ■ main_runner.py       # Orquestador del pipeline
■ ■ ■ ■ ■ sorting/            # 12 algoritmos de ordenamiento
■ ■ ■ ■ ■ __init__.py
■ ■ ■ ■ ■ algorithms.py
■ ■ ■ ■ ■ comparator.py
■ ■ ■ ■ ■ visualizer.py
■ ■ ■ ■ ■ static/
■ ■ ■ ■ ■ css/style.css        # Estilos del frontend
■ ■ ■ ■ ■ js/charts.js         # Utilidades Chart.js
■ ■ ■ ■ ■ tests/
■ ■ ■ ■ ■ test_similarity.py    # 20 tests de similitud
■ ■ ■ ■ ■ test_patterns.py     # 21 tests de patrones y volatilidad
■ ■ ■ ■ ■ test_dashboard.py    # 6 tests de SMA y dashboard
■ ■ ■ ■ ■ data/
■ ■ ■ ■ ■ raw/
■ ■ ■ ■ ■ processed/
■ ■ ■ ■ ■ docs/
■ ■ ■ ■ ■ Diseno.md            # Este documento
■ ■ ■ ■ ■ Documento de diseño.docx
■ ■ ■ ■ ■ outputs/            # Reportes PDF generados
■ ■ ■ ■ ■ Proyecto.md          # Enunciado del proyecto
■ ■ ■ ■ ■ Plan.md              # Plan de desarrollo
■ ■ ■ ■ ■ requirements.txt
■ ■ ■ ■ ■ Taskfile.yml
■ ■ ■ ■ ■ docker-compose.yml
■ ■ ■ ■ ■ Dockerfile.api
■ ■ ■ ■ ■ Dockerfile.etl

```

2.2 Flujo de Ejecución

Extracción (fetcher) → Limpieza (cleaner) → Unificación (unifier)



2.3 Stack Tecnológico

Capa	Tecnología	Propósito
Backend	Python 3.11+	Lenguaje principal
API	Flask 3.0	Framework web REST
Frontend	Jinja2 + Chart.js	Templates y gráficos interactivos
HTTP	requests 2.31	Peticiones HTTP directas a APIs

PDF	ReportLab 4.1	Generación de reportes
Gráficos PDF	Matplotlib 3.8	Imágenes para reportes
Testing	pytest 7.4	Tests unitarios
Linter	Ruff 0.1	Calidad de código
Infra	Docker + Taskfile	Contenedores y automatización

3. Requerimiento 1 — ETL Automatizado

3.1 Extracción (peticiones HTTP directas)

Se utiliza Yahoo Finance API mediante HTTP directo, cumpliendo con la restricción de no usar librerías de alto nivel como *yfinance*.

Construcción de la consulta:

```
GET https://query1.finance.yahoo.com/v8/finance/chart/{SIMBOL0}
?period1={TIMESTAMP_INICIO}
&period2;={TIMESTAMP_FIN}
&interval;=1d
```

Parsing manual: Se extraen los arrays *timestamp* e *indicators.quote* de la respuesta JSON, iterando sobre cada posición para construir registros OHLCV (Open, High, Low, Close, Volume).

Manejo de errores: Reintentos automáticos (MAX_RETRIES = 3) con backoff exponencial (2s, 4s, 6s).

20 activos: 4 acciones colombianas (ECOPETROL.CL, ISA.CL, GEB.CL, NUTRESA.CL) + 16 ETFs internacionales (VOO, VTI, QQQ, SPY, VEA, VWO, BND, EFA, EEM, TLT, IVV, SCHD, DIA, IWM, XLF, XLK).

3.2 Limpieza de Datos

Eliminación de Duplicados

- **Técnica:** HashSet con tuplas (date, symbol) como clave
- **Complejidad:** O(n) tiempo, O(n) espacio
- **Justificación:** Cada par (fecha, símbolo) debe ser único. Los duplicados distorsionan estadísticas (media, varianza, volumen total).

Detección de Outliers — Z-Score

- **Fórmula:** $|z| = |(x - \mu) / \sigma| > 3.0$
- **Complejidad:** O(n) — dos pasadas (media y desviación)
- **Limitación:** Asume distribución normal de los datos

Detección de Outliers — IQR (Rango Inter cuartil)

- **Fórmula:** Outlier si $x < Q1 - 1.5 \times IQR$ o $x > Q3 + 1.5 \times IQR$
- **Complejidad:** O(n log n) — dominada por el ordenamiento para cuartiles
- **Justificación:** Más robusto que Z-Score porque no asume distribución normal. Los retornos financieros típicamente no siguen una distribución normal (tienen colas pesadas).

Interpolación Lineal

- **Fórmula:** valor = (anterior + siguiente) / 2
- **Complejidad:** O(n) — búsqueda de vecinos + promedio
- **Justificación:** Preserva la longitud del dataset y mantiene tendencias sin discontinuidades.

Forward Fill

- **Técnica:** Propagar el último valor conocido hacia adelante
- **Complejidad:** $O(n)$
- **Aplicación:** Cuando el valor se mantiene constante hasta nuevo registro

Backward Fill

- **Técnica:** Propagar el siguiente valor conocido hacia atrás
- **Complejidad:** $O(n)$
- **Aplicación:** Valores faltantes al inicio de la serie

Orden del pipeline de limpieza:

1. Duplicados primero (afectan estadísticas)
 2. Outliers después (basados en estadísticas corregidas)
 3. Interpolación último (usa contexto temporal completo)
- Lineal primero (mejor estimación con vecinos)
 - Forward-fill segundo (respaldo si falta vecino anterior)
 - Backward-fill último (respaldo si falta vecino siguiente)

3.3 Alineación de Calendarios Bursátiles

Se implementó un sistema de detección de días hábiles para cada mercado:

- **BVC (Colombia):** 18 festivos fijos por año + fines de semana
- **NYSE (EE.UU.):** 10 festivos fijos por año + fines de semana

Algoritmo de alineación:

1. Para cada símbolo, se identifica su mercado (BVC o NYSE)
2. Se calcula la unión de todos los días hábiles de ambos mercados
3. Para fechas donde un activo no tiene datos, se inserta un registro con valores *None*
4. El *DataCleaner* interpola esos valores en la siguiente etapa

Complejidad: $O(s \times d + n)$ donde s = símbolos, d = días en el rango, n = registros

4. Requerimiento 2 — Algoritmos de Similitud

4.1 Distancia Euclidiana

Archivo: `src/services/similarity/euclidean.py`

Fórmula:

$$d(x, y) = \sqrt{\left(\frac{1}{n} \times \sum (x_i - y_i)^2 \right)}$$

Interpretación: 0 = series idénticas. Valores más altos = mayor disimilitud.

Complejidad: $O(n)$ tiempo, $O(1)$ espacio — una pasada para diferencias al cuadrado.

Aplicación: Comparación de precios normalizados o retornos diarios.

4.2 Correlación de Pearson

Archivo: `src/services/similarity/pearson.py`

Fórmula:

$$r = \frac{\sum ((x_i - \bar{x})(y_i - \bar{y}))}{\sqrt{(\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2)}}$$

Interpretación: $r \in [-1, 1]$. 1 = correlación perfecta positiva, 0 = sin relación lineal, -1 = correlación inversa.

Complejidad: $O(n)$ tiempo, $O(1)$ espacio — media (1 pasada) + covarianza/varianzas (1 pasada).

Aplicación: Medir relación lineal entre retornos de activos para diversificación de portafolio.

4.3 Dynamic Time Warping (DTW)

Archivo: `src/services/similarity/dtw.py`

Fórmula:

$$DTW(x, y) = \min_{\pi} \sum_{(i,j) \in \pi} |x_i - y_j|$$

Donde π es un camino de warping que alinea los índices de ambas series.

Algoritmo: Programación dinámica — matriz de distancia acumulada D de tamaño $(n+1) \times (m+1)$:

$$D[i][j] = |x_i - y_j| + \min(D[i-1][j], D[i][j-1], D[i-1][j-1])$$

Optimización (Sakoe-Chiba): Restringe el warping a una banda de ancho w alrededor de la diagonal, reduciendo la complejidad de $O(n \times m)$ a $O(n \times w)$.

Complejidad: $O(n \times m)$ sin optimización, $O(n \times w)$ con Sakoe-Chiba.

Aplicación: Comparar activos con desfases temporales o diferentes calendarios.

4.4 Similitud por Coseno

Archivo: `src/services/similarity/cosine.py`

Fórmula:

$$\cos(\theta) = (x \cdot y) / (||x|| \cdot ||y||) = \sum(x_i \cdot y_i) / \sqrt{\sum x_i^2} \cdot \sqrt{\sum y_i^2}$$

Interpretación: $\cos(\theta) \in [-1, 1]$. 1 = misma dirección, 0 = ortogonal, -1 = dirección opuesta. A diferencia de Pearson, NO centra los datos (no resta la media).

Complejidad: $O(n)$ tiempo, $O(1)$ espacio — una pasada para producto punto y normas.

Aplicación: Comparar perfiles de retornos diarios considerando magnitud y dirección.

4.5 SimilarityAnalyzer (Agregador)

Archivo: `src/services/similarity/__init__.py`

La clase *SimilarityAnalyzer* coordina los 4 algoritmos:

1. Extrae y alinea series de dos símbolos por fecha común ($O(n)$)
2. Calcula retornos diarios ($O(n)$)
3. Ejecuta los 4 algoritmos y retorna resultados estructurados
4. Método `compute_correlation_matrix()` para matriz completa $O(s^2 \times n)$

Endpoint API: `GET /api/similarity?s1=V00&s2=SPY`

5. Requerimiento 3 — Patrones y Volatilidad

5.1 Ventana Deslizante

Archivo: `src/services/patterns/sliding_window.py`

Patrón 1: Días Consecutivos al Alza

- **Definición:** Secuencia de N días donde close > close anterior
- **Algoritmo:** Contador acumulado — O(n) tiempo, O(1) espacio

...

contador = 0

para cada día i:

si close[i] > close[i-1]: contador++

sino: contador = 0

si contador >= N: patrón detectado

...

- **Endpoint:** *GET /api/patterns?symbol=V00&pattern;=consecutive_up&min;_days=3*

Patrón 2: Gap Up

- **Definición:** Día donde open > prev_close × (1 + threshold)
- **Algoritmo:** Comparación día contra día anterior — O(n)
- **Parámetro:** threshold configurable (default 2%)
- **Endpoint:** *GET /api/patterns?symbol=V00&pattern;=gap_up&threshold;=0.02*

5.2 Volatilidad y Clasificación de Riesgo

Archivo: *src/services/patterns/volatility.py*

Cálculo de Retornos Diarios

```
r[i] = (close[i] - close[i-1]) / close[i-1]
```

Complejidad: O(n)

Desviación Estándar

```
σ = √((Σ(r[i] - r_bar)²) / (n-1))
```

Complejidad: O(n)

Volatilidad Anualizada

```
σ_anual = σ_diaria × √252
```

Se usan 252 días de negociación como estándar bursátil.

Clasificación de Riesgo

Categoría	Rango	Color
Conservador	$\sigma_{\text{anual}} < 15\%$	■ Verde
Moderado	$15\% \leq \sigma_{\text{anual}} < 30\%$	■ Amarillo
Agresivo	$\sigma_{\text{anual}} \geq 30\%$	■ Rojo

Complejidad total: O(s × n + s log s) donde s = número de símbolos, n = registros por símbolo.

Endpoints:

- *GET /api/volatility?symbol=V00* — Métricas individuales
- *GET /api/volatility/ranking* — Ranking completo

6. Requerimiento 4 — Dashboard y Reportes

6.1 Matriz de Correlación

Calcula el coeficiente de Pearson entre todos los pares de activos.

- **Complejidad:** $O(s^2 \times n)$ donde s = símbolos, n = observaciones
- **Visualización:** Heatmap dinámico en el frontend (colores por valor)
- **Endpoint:** `GET /api/correlation-matrix`

6.2 Media Móvil Simple (SMA)

Archivo: `src/services/reporting/technical.py`

```
SMA[i] = (cumsum[i] - cumsum[i - window]) / window
```

- SMA-20: media de 20 días (~1 mes de negociación)
- SMA-50: media de 50 días (~2.5 meses)
- **Complejidad:** $O(n)$ con suma acumulativa

Endpoint: `GET /api/candlestick?symbol=V00&smas;=20,50&limit;=252`

6.3 Reporte PDF

Archivo: `src/services/reporting/pdf_report.py`

Tecnología: ReportLab para estructura + matplotlib para gráficos incrustados.

Secciones del PDF:

1. **Portada:** Universidad, curso, título, fecha de generación
2. **Resumen del Portafolio:** activos, registros, rango de fechas
3. **Matriz de Correlación:** heatmap generado con matplotlib, tabla de pares extremos
4. **Ranking de Riesgo:** tabla completa con colores por categoría, distribución
5. **Candlestick:** gráfico de precios con SMA-20 y SMA-50
6. **Similitud:** tabla con las 4 métricas del par más correlacionado

Endpoint: `POST /api/report/generate` (con JSON opcional `{"symbol": "V00"}`)

7. Requerimiento 5 — Despliegue y Documentación

7.1 API REST

La API está organizada en Blueprints de Flask para modularidad:

Blueprint	Endpoints	Propósito
`similarity_bp`	<code>GET /api/similarity`</code>	Similitud entre 2 activos
	<code>GET /api/similarity/symbols`</code>	Listar símbolos disponibles
	<code>GET /api/correlation-matrix`</code>	Matriz de correlación completa
`patterns_bp`	<code>GET /api/patterns`</code>	Detección de patrones
	<code>GET /api/volatility`</code>	Volatilidad por activo
	<code>GET /api/volatility/ranking`</code>	Ranking de riesgo
`dashboard_bp`	<code>GET /api/candlestick`</code>	Datos OHLC + SMA

	`GET /api/dashboard/summary`	Resumen del dashboard
`reports_bp`	`POST /api/report/generate`	Generar PDF

Además, rutas HTML para las páginas del frontend:

- `/` — Inicio
- `/similarity` — Página de similitud interactiva
- `/patterns` — Página de patrones
- `/risk` — Página de riesgo
- `/dashboard` — Dashboard completo

7.2 Frontend

Tecnología: Jinja2 (templates Flask) + Chart.js (visualizaciones).

Páginas:

- **Inicio:** Cards resumen con estadísticas del dataset, tabla de activos
- **Similitud:** Selector de 2 activos, gráfica superpuesta, tabla de 4 métricas, explicación matemática
- **Patrones:** Selector de activo + patrón, gráfica de frecuencia por año, últimas ocurrencias
- **Riesgo:** Ranking ordenado con colores, gráfico doughnut de distribución, stats resumen
- **Dashboard:** Heatmap de correlación dinámico, candlestick con toggle SMA-20/SMA-50, botón de exportación PDF

7.3 Despliegue

Docker:

```
docker-compose up # Inicia API + ETL
```

Taskfile:

```
task install # Crear entorno virtual + instalar dependencias
task run     # Pipeline ETL completo
task api     # Servidor web en localhost:5000
task test    # Ejecutar tests
task lint    # Verificar código
```

7.4 Reproducibilidad

El pipeline es completamente reproducible:

```
task install
python -m src.services.main_runner --force-download
python -m src.api.gateway
```

El flag `--force-download` garantiza que los datos se descarguen desde cero, sin usar caché.

8. Análisis de Complejidad Consolidado

Componente	Algoritmo	Complejidad Temporal	Complejidad Espacial
ETL - Descarga	HTTP requests paralelizados	$O(a)$ requests	$O(n \times a)$ datos
ETL - Duplicados	HashSet	$O(n)$	$O(n)$
ETL - Outliers Z-Score	Media + Desviación	$O(n)$	$O(1)$
ETL - Outliers IQR	Ordenar + Cuartiles	$O(n \log n)$	$O(n)$

ETL - Interpolación Lineal	Vecinos + Promedio	$O(n)$	$O(n)$
ETL - Forward Fill	Propagación adelante	$O(n)$	$O(1)$
ETL - Backward Fill	Propagación atrás	$O(n)$	$O(1)$
ETL - Alineación Calendarios	Unión de fechas	$O(s \times d)$	$O(d)$
ETL - Unificación	Sort + Merge	$O(n \log n)$	$O(n)$
Similitud - Euclidiana	Suma diferencias ²	$O(n)$	$O(1)$
Similitud - Pearson	Covarianza / Varianza	$O(n)$	$O(1)$
Similitud - DTW	Programación Dinámica	$O(n \times m) / O(n \times w)$	$O(n \times m)$
Similitud - Coseno	Producto punto	$O(n)$	$O(1)$
Similitud - Matriz Correlación	Pearson por pares	$O(s^2 \times n)$	$O(s^2)$
Patrones - Consecutive Up	Contador acumulado	$O(n)$	$O(1)$
Patrones - Gap Up	Comparación día anterior	$O(n)$	$O(1)$
Volatilidad	Desviación estándar	$O(n)$	$O(1)$
SMA	Suma acumulativa	$O(n)$	$O(n)$
Reporte PDF	Generación + matplotlib	$O(n)$	$O(n)$

Donde:

- n = número de registros por activo (≈ 1250 para 5 años)
- s = número de símbolos (20)
- a = número de activos (20)
- d = días en el rango temporal
- m = longitud de segunda serie (DTW)
- w = ancho de banda Sakoe-Chiba

9. Tests Unitarios

Total: 47 tests, todos pasando.

Archivo	Tests	Cobertura
`tests/test_similarity.py`	20	Distancia Euclidiana, Pearson, DTW, Coseno, SimilarityAnalyzer
`tests/test_patterns.py`	21	Consecutive Up, Gap Up, PatternAnalyzer, retornos, desviación, volatilidad, clasificación, VolatilityAnalyzer, ranking
`tests/test_dashboard.py`	6	SMA con diferentes ventanas, casos borde

Ejecución: `task test o python -m pytest tests/ -v`

10. Declaración de Uso de IA

En el desarrollo de este proyecto se utilizó inteligencia artificial generativa como apoyo para:

Herramientas utilizadas

- **Anthropic Claude (CLI de opencode):** Asistente principal para planificación, generación de código y resolución de problemas

Alcance del uso

1. **Planificación:** Estructuración inicial del proyecto en fases (*Plan.md*)
2. **Generación de código base:** Implementación de los 4 algoritmos de similitud, detección de patrones, módulo de volatilidad, dashboard y reporte PDF
3. **Refactorización:** Migración de rutas planas a Blueprints de Flask
4. **Frontend:** Generación de templates HTML con Chart.js
5. **Tests:** Creación de tests unitarios para cada módulo
6. **Documentación:** Este documento de diseño

Lo que NO fue generado por IA

- El diseño algorítmico y la selección de técnicas
- El análisis formal de complejidad computacional (Big-O)
- La fundamentación matemática de cada algoritmo
- Las decisiones de diseño arquitectónico
- La interpretación de resultados financieros

Transcripción parcial de prompts relevantes

1. **"Crear un plan detallado para implementar un sistema de análisis algorítmico financiero con ETL, similitud, patrones, dashboard y PDF"**
2. **"Implementar algoritmo de Dynamic Time Warping con optimización Sakoe-Chiba para series financieras"**
3. **"Generar reportead PDF con ReportLab que incluya heatmap de correlación y ranking de riesgo"**
4. **"Crear tests unitarios para los 4 algoritmos de similitud con datos sintéticos conocidos"**

11. Limitaciones y Trabajo Futuro

Limitaciones conocidas

- La API de Yahoo Finance puede tener rate limiting (500 peticiones por hora)
- Los festivos colombianos se modelan con fechas fijas (no móviles como Semana Santa)
- El DTW sin restricción de ventana tiene complejidad $O(n^2)$ para series largas
- La generación de PDF usa archivos temporales para imágenes matplotlib

Posibles mejoras

- Implementar caché de datos con SQLite para evitar descargas repetidas
- Agregar autenticación básica a la API
- Soporte para más mercados (Europa, Asia)
- Patrones adicionales: doble techo, cabeza y hombros, banderas
- Notificaciones por email cuando se detecten patrones
- Despliegue en la nube (Heroku, Railway, o servidor VPS)

Documento de diseño generado para el curso de Análisis de Algoritmos — Universidad del Quindío — 2026-1